Python-Selenium

一，全面讲解 XPATH（常用定位方式）

XPath 是 Selenium 中用于定位页面元素的语法，分为**绝对路径**和**相对路径**两类：

1. 绝对路径（以 `/` 开头）

从 HTML 根标签开始，逐层写全所有标签的路径，定位到目标元素（不推荐，页面结构变化会导致定位失效）。示例：

```
/html/body/form/table/tbody/tr[2]/table/tbody/tr[1]/td[2]/input
```

（说明：从根标签 `<html>` 开始，依次定位到 `body` → `form` → `table` →第 2 个 `tr` →嵌套的 `table` →第 1 个 `tr` →第 2 个 `td` 下的 `input` 元素）

2. 相对路径（以 `//` 开头）

不依赖根标签，直接从匹配的元素开始定位（推荐，更灵活、稳定），包含多种定位方式：

（1）相对路径 + 索引定位

通过 “标签 + 索引” 定位（索引从 1 开始），适合同标签有多个的场景。示例：

```
//input[2]
```

（说明：定位页面中**第 2 个 `<input>` 标签**的元素）

（2）相对路径 + 属性定位

通过元素的属性（如 `name` / `type` / `value` 等）定位，是最常用的方式之一。

- **单属性定位**：示例：

  ```
  //input[@name="username"]  <!-- 定位name属性为"username"的<input>元素 -->
  //input[@type="submit"]    <!-- 定位type属性为"submit"的<input>元素 -->
  ```

- **多属性定位**：同时用多个属性缩小范围（更精准，避免属性重复）。示例：

  ```
  //input[@type="submit" and @value="进入管理中心"]
  ```

  （说明：定位 `type="submit"` **且** `value="进入管理中心"` 的 `<input>` 元素）

（3）相对路径 + 部分属性定位

通过属性的 “部分内容” 定位，适用于属性值较长 / 动态变化的场景。

- `starts-with`：匹配 “属性值以某内容开头” 的元素示例：

  ```
  //input[starts-with(@value,"进入")]
  ```

（说明：定位`value`属性**以“进入”开头**的`<input>`元素）

- `contains`：匹配“属性值包含某内容”的元素示例：

```
//input[contains(@value,"管理")]
```

（说明：定位`value`属性 ** 包含“管理”** 的`<input>`元素）

（4）相对路径 + 文本值定位

通过元素的**显示文本**定位（常用于链接、按钮等元素），可替代 Selenium 的`LINK_TEXT`/`PARTIAL_LINK_TEXT`。

- 精确匹配文本：示例：

```
//a[text()="商品列表"]
```

（说明：定位文本 ** 完全等于“商品列表”** 的`<a>`标签元素）

- 模糊匹配文本（结合`contains`）：示例：

```
//a[contains(text(),"商品")]
```

（说明：定位文本 ** 包含“商品”** 的`<a>`标签元素）

（5）相对路径 + 通配符定位

用`*`表示“任意标签”，适用于不确定标签名的场景。示例：

```
//*[text()="商品列表"]  <!-- 定位文本为“商品列表”的任意标签元素 -->
//*[@type="submit"]      <!-- 定位type属性为“submit”的任意标签元素 -->
```

补充说明

- 绝对路径**不推荐**：页面结构（比如新增 / 删除标签）变化时，绝对路径会直接失效；
- 相对路径优先用**属性 / 文本定位**：更稳定，受页面结构影响小；
- 多属性 / 部分属性定位：能提高定位的唯一性，减少元素重复导致的定位错误。

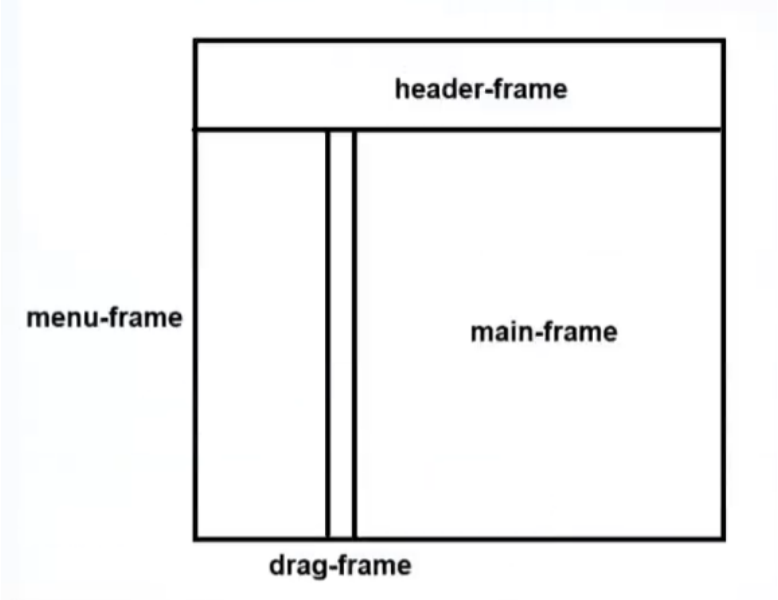
简单应用

```
import time  # 导入时间模块，用于添加等待（可选）
from selenium import webdriver  # 导入selenium的webdriver，用于控制浏览器
from selenium.webdriver.common.by import By  # 导入By类，用于指定元素定位方式（如XPATH）


def test_login():
    # 1. 配置Chrome浏览器选项：解决浏览器执行完用例后自动关闭的问题
    options = webdriver.ChromeOptions()  # 创建Chrome浏览器的配置对象
    # 添加实验性选项：detach=True 表示浏览器在脚本执行完后保持打开
    options.add_experimental_option(name="detach", value=True)

    # 2. 创建Chrome浏览器对象（传入配置选项）
    driver = webdriver.Chrome(options=options)

    # 3. 加载目标页面（这里是ECShop的后台登录页）
```

```
    driver.get("http://192.168.0.44/ecshop/admin/privilege.php?act=logout")

    # 4. 定位“用户名输入框”，并输入用户名“admin”
    # 定位方式：XPATH，匹配name属性为"username"的input元素
    driver.find_element(By.XPATH,
value="//input[@name='username']").send_keys("admin")

    # 5. 定位“密码输入框”，并输入密码“admin123”
    # 定位方式：XPATH，匹配name属性为"password"的input元素
    driver.find_element(By.XPATH,
value="//input[@name='password']").send_keys("admin123")

    # 6. 定位“登录按钮”，并点击
    # 定位方式：XPATH，匹配value属性为"进入管理中心"的input元素
    driver.find_element(By.XPATH, value="//input[@value='进入管理中心']").click()
```

二，实战应用

```
项目登陆之后一定要特别注意是否有frame和iframe框架。
frame：框架（房间）：如果元素在房间里面，那么就必须先进入框架才能定位。
iframe：子框架（房间）：如果元素在房间里面，那么就必须先进入框架才能定位。
```



1. 框架（frame）处理代码

```
# 1. 进入指定框架（frame）
# 说明：切换到name为"menu-frame"的框架内，后续才能定位该框架中的元素
driver.switch_to.frame("menu-frame")

# 2. 退出框架（回到主页面）
# 说明：从当前框架切换回主文档，之后可以操作主页面或其他框架的元素
driver.switch_to.default_content()
```

2. 下拉框（select）处理代码

```
# 先导入Select类（需补充这行代码，否则会报错）
from selenium.webdriver.support.select import Select
```

```
# 1. 定位下拉框元素
# 说明：通过XPATH定位name属性为"cat_id"的<select>标签（即下拉框本身）
ele = driver.find_element(By.XPATH, "//select[@name='cat_id']")

# 2. 初始化Select对象（将定位到的下拉框元素传入Select，用于操作下拉选项）
sel = Select(ele)

# 3. 下拉框选择方式1：通过索引选择（索引从0开始）
# 说明：选择下拉框中第3个选项（索引2对应第3个）
# sel.select_by_index(2)

# 4. 下拉框选择方式2：通过选项的value属性选择
# 说明：选择value属性值为"2"的选项
# sel.select_by_value("2")

# 5. 下拉框选择方式3：通过选项的显示文本选择（最常用，直观）
# 说明：选择下拉框中显示文本为“手机类型”的选项
sel.select_by_visible_text("手机类型")
```

关键补充说明

1. **框架处理的核心逻辑**：
    - 进入框架后，只能操作该框架内的元素；
    - 操作完必须退出框架（`default_content()`），才能操作主页面 / 其他框架的元素。
2. **下拉框处理的前提**：
    - 必须先导入`Select`类（代码中第一行的`from selenium.webdriver.support.select import Select`）；
    - 仅适用于 HTML 中`<select>`标签的下拉框（自定义下拉框需用其他方式操作）。
3. **下拉框 3 种选择方式的场景**：
    - `select_by_index`：知道选项的顺序（但页面选项顺序变化会失效）；
    - `select_by_value`：知道选项的`value`属性值（适合选项文本动态变化的场景）；
    - `select_by_visible_text`：知道选项的显示文本（最直观，常用）。

3.网页新增过程中，元素特殊属性的操作

在网页中，部分元素会有**特殊属性**限制操作（如只读、不可见、不可编辑），需要先移除这些属性才能交互：

- `readonly`：元素只读（无法直接输入内容，如日历输入框）
- `style="display:none"`：元素不可见（无法定位 / 操作）
- `disabled`：元素不可编辑（无法点击 / 输入）

移除`readonly`属性

```
# 需提前导入的模块（代码中可能省略了，需补充）
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

def test_handle_readonly_calendar():
```

```
    # 1. 定位“只读的日历输入框”
    # 说明：通过XPATH定位name属性为"promote_end_date"的输入框（该输入框默认是readonly只读状态）
    ele = driver.find_element(By.XPATH, "//input[@name='promote_end_date']")

    # 2. 执行JavaScript代码，移除输入框的"readonly"属性
    # 说明：Selenium无法直接修改元素属性，需通过execute_script执行JS来移除限制
    # arguments[0]：代表传入的第一个参数（即上面定位到的ele元素）
    # removeAttribute('readonly')：JS方法，移除元素的readonly属性
    driver.execute_script("arguments[0].removeAttribute('readonly');", ele)

    # 3. 清空输入框原有内容（移除只读后才能操作）
    ele.clear()

    # 4. 输入目标日期（如"2025-05-05"）
    ele.send_keys("2025-05-05")

    # 5. 等待5秒，查看操作结果（可选，用于调试）
    time.sleep(5)
```

关键注释说明

1. **为什么需要执行 JS?** Selenium 是 “模拟用户操作”，而 `readonly` 是元素的**HTML 属性限制**（用户也无法直接输入），所以必须通过 JavaScript 修改元素属性，绕过这个限制。
2. **代码核心逻辑**：定位只读元素 → 用 JS 移除 `readonly` 属性 → 正常操作输入框（清空 + 输入）。
3. **其他特殊属性的处理思路**：
    - 移除 `disabled`（不可编辑）：把 JS 改为 `arguments[0].removeAttribute('disabled');`
    - 显示 `display:none`（不可见）：把 JS 改为 `arguments[0].style.display='block';`

4. 文件上传

文件上传的核心逻辑：只要是 HTML 原生的 `<input type="file">` 上传控件，都可以直接用 `send_keys(文件路径)` 完成上传，无需模拟 “点击选择文件→选择本地文件” 的操作（这是 Selenium 处理文件上传的最简方式）。

```
# 1. 定位文件上传输入框，并直接传入文件路径
# 说明：
# - 文件上传的<input>标签（type通常为"file"）支持直接用send_keys传入文件路径，无需点击“选择文件”按钮
# - r"D:\workspace\pythonwork\web_f...": r代表“原始字符串”，避免路径中的\被识别为转义符
# - 需替换为实际的文件路径（如图片、文档的本地路径）
driver.find_element(By.XPATH,
"//input[@name='goods_img']").send_keys(r"D:\workspace\pythonwork\web_f\目标文件.png")
```

5. Alert 弹窗处理代码

Alert 弹窗的注意事项：

- 仅适用于浏览器**原生弹窗**（不是网页自定义的弹窗）；
- 操作完弹窗后，会自动回到主页面上下文；

- `accept()` 对应 “确定”，`dismiss()` 对应 “取消”。

```
# 1. 切换到Alert弹窗（Selenium默认在主页面，需先切换到弹窗上下文）
# 说明：Alert是浏览器原生弹窗（如提示框、确认框），必须先切换才能操作
ale = driver.switch_to.alert

# 2. 点击弹窗的“确定”按钮（接受弹窗）
ale.accept()

# 3. （可选）点击弹窗的“取消”按钮（关闭弹窗，该行被注释）
# ale.dismiss()
```

三，引入Pytest管理测试用例

Pytest是基于Python的开源测试框架，语法简单易用，有大量的插件，功能非常多。自动检测测试用例，支持参数化，跳过特定用例，失败重试等功能。

1. 安装

使用pip命令安装：

```
pip install -U pytest  # 安装
pytest --version # 查看版本
pytest -h # 查看帮助信息
```

2.用例识别

- pytest识别文件名为test.*py或者*test.py的测试文件
- 测试文件中可以在Test*类中写测试用例（测试用例以test*_开头，并且测试类不能有**init**方法）
- 不在Test*类中的test*_用例也可以被识别到。

打印测试目录下匹配到的所有用例：`pytest --collect-only`

3. 常用选项

参数	作用 & 示例
`-v, --version`	查看当前 pytest 的版本信息示例：`pytest --version`
`-h, --help`	查看 pytest 所有命令行参数的帮助说明示例：`pytest --help`
`-k EXPRESSION`	只运行**名称匹配 EXPRESSION**的用例（支持模糊匹配）示例：`pytest -k "add"` → 运行文件名 / 用例名包含`add`的用例
`-m MARKEPR`	只运行**标记为 MARKEPR**的用例（需提前用`@pytest.mark.标记名`标记用例）示例：`pytest -m login` → 运行带`login`标记的用例
`-x, --exitfirst`	遇到**第一个失败用例时立即停止**执行（适合快速定位首个错误）示例：`pytest -x`
`--maxfail=num`	当失败用例数达到`num`时停止执行（比`-x`更灵活）示例：`pytest --maxfail=2` → 失败 2 个用例后停止
`--lf, --last-failed`	只运行**上一次执行失败的用例**（失败用例记录在`.pytest_cache/`目录中）示例：`pytest --lf`
`--ff, --failed-first`	先运行上一次失败的用例，再运行其他用例（优先验证失败用例是否修复）示例：`pytest --ff`
`-s`	在命令行显示代码中`print()`的内容（默认 pytest 会屏蔽`print`输出）示例：`pytest -s`
`-v, --verbose`	打印**详细的用例执行日志**（包含用例路径、执行结果等）示例：`pytest -v`（常和`s`组合用：`pytest -vs`）
`-q, --quiet`	打印**简略的用例执行日志**（只显示结果统计，不显示单个用例详情）示例：`pytest -q`
`--collect-only, --co`	只收集所有测试用例（不实际执行），用于检查用例是否被正确识别示例：`pytest --collect-only`

4. Pytest用例

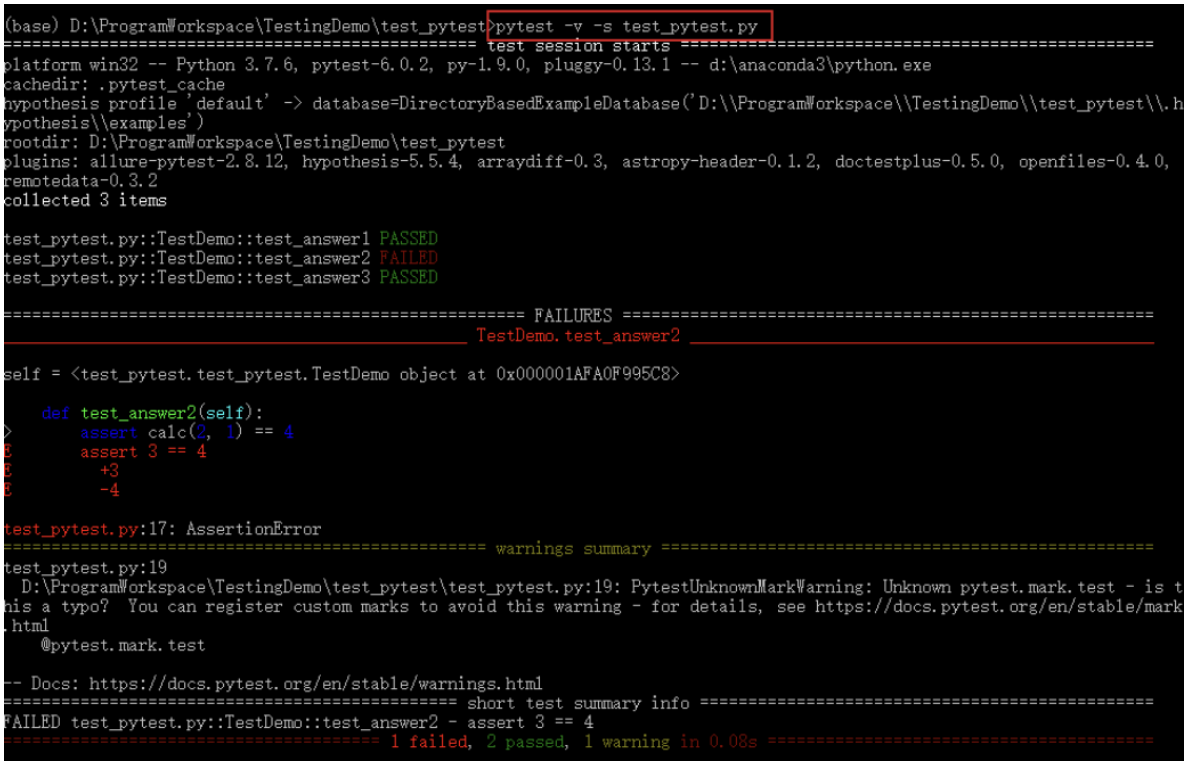
```
import pytest

def calc(a,b):
    return a + b

class TestDemo():
    def test_answer1(self):
        assert calc(1, 1) == 2

    def test_answer2(self):
        assert calc(2, 1) == 3

  @pytest.mark.answer3
    def test_answer3(self):
        assert calc(6, 6) == 12

if __name__=='__main__':
    pytest.main()
```

4.1.执行用例

```
# 执行test_pytest.py所有用例（模块）
pytest test_pytest.py

# 执行test_pytest.py里的TestDemo类
pytest test_pytest.py::TestDemo

# 执行test_pytest.py里的TestDemo类的test_answer2方法
pytest test_pytest.py::TestDemo::test_answer2
```

4.2. 打印日志信息

```
# 打印详细运行日志信息
pytest -v test_pytest.py
pytest -s test_pytest.py
```



4.3. 跳过某个用例

```
# 跳过运行某个用例
pytest -v -k "类名 and not方法名" 文件名
pytest -v -k "TestDemo and not test_answer2" test_pytest.py
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v -k "TestDemo and not test_answer2" test_pytest.py
============================================= test session starts =============================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.h
ypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0,
remotedata-0.3.2
collected 3 items / 1 deselected / 2 selected

test_pytest.py::TestDemo::test_answer1 PASSED                                                            [ 50%]
test_pytest.py::TestDemo::test_answer3 PASSED                                                            [100%]

=============================================== warnings summary ==============================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.test - is t
his a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/mark
.html
    @pytest.mark.test

-- Docs: https://docs.pytest.org/en/stable/warnings.html
=================================== 2 passed, 1 deselected, 1 warning in 0.04s ====================================
```

4.4. 运行文件名包含某个关键字的用例

识别包含“add”的用例：`pytest -k "add" --collect-only`

运行包含“add”的用例：`pytest -k "add" -vs`

4.5. 运行某个标记的用例

```
# 将运行有这个标记的测试用例：@pytest.mark.[标记名]
pytest -m [标记名] 文件名
pytest -m answer3 test_pytest.py
```

4.6. 报错停止

```
# 一旦运行到报错用例就停止运行
pytest -x 文件名
pytest -x test_pytest.py

# 当报错达到num的时候就停止运行
pytest --maxfail=[num] 文件名
pytest --maxfail=1 test_pytest.py
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v -x test_pytest.py
========================================== test session starts ==========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.h
ypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0,
remotedata-0.3.2
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED                                                          [ 33%]
test_pytest.py::TestDemo::test_answer2 FAILED                                                          [ 66%]

============================================== FAILURES ==============================================
_____________________________________________ TestDemo.test_answer2 _____________________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x000002347324D5C8>

    def test_answer2(self):
>       assert calc(2, 1) == 4
E       assert 3 == 4
E         +3
E         -4

test_pytest.py:17: AssertionError
========================================== warnings summary ==========================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - i
s this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/m
ark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
========================================== short test summary info ==========================================
FAILED test_pytest.py::TestDemo::test_answer2 - assert 3 == 4
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
=================================== 1 failed, 1 passed, 1 warning in 0.12s ===================================
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v --maxfail=1 test_pytest.py
========================================== test session starts ==========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.h
ypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0,
remotedata-0.3.2
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED                                                          [ 33%]
test_pytest.py::TestDemo::test_answer2 FAILED                                                          [ 66%]

============================================== FAILURES ==============================================
_____________________________________________ TestDemo.test_answer2 _____________________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x0000024491E0C648>

    def test_answer2(self):
>       assert calc(2, 1) == 4
E       assert 3 == 4
E         +3
E         -4

test_pytest.py:17: AssertionError
========================================== warnings summary ==========================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - i
s this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/m
ark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
========================================== short test summary info ==========================================
FAILED test_pytest.py::TestDemo::test_answer2 - assert 3 == 4
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
=================================== 1 failed, 1 passed, 1 warning in 0.11s ===================================
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v -x test_pytest.py
========================================= test session starts =========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.h
ypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0,
remotedata-0.3.2
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED                                                    [ 33%]
test_pytest.py::TestDemo::test_answer2 FAILED                                                    [ 66%]

========================================== FAILURES ===========================================
_____________________________________ TestDemo.test_answer2 _____________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x000002347324D5C8>

    def test_answer2(self):
>       assert calc(2, 1) == 4
E       assert 3 == 4
E         +3
E         -4

test_pytest.py:17: AssertionError
========================================= warnings summary =========================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - i
s this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/m
ark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===================================== short test summary info =====================================
FAILED test_pytest.py::TestDemo::test_answer2 - assert 3 == 4
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
=================================== 1 failed, 1 passed, 1 warning in 0.12s ====================================
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v --maxfail=1 test_pytest.py
========================================= test session starts =========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.h
ypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0,
remotedata-0.3.2
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED                                                    [ 33%]
test_pytest.py::TestDemo::test_answer2 FAILED                                                    [ 66%]

========================================== FAILURES ===========================================
_____________________________________ TestDemo.test_answer2 _____________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x0000024491E0C648>

    def test_answer2(self):
>       assert calc(2, 1) == 4
E       assert 3 == 4
E         +3
E         -4

test_pytest.py:17: AssertionError
========================================= warnings summary =========================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - i
s this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/m
ark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
===================================== short test summary info =====================================
FAILED test_pytest.py::TestDemo::test_answer2 - assert 3 == 4
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! stopping after 1 failures !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
=================================== 1 failed, 1 passed, 1 warning in 0.11s ====================================
```

4.7 失败重新运行

安装 pytest-rerunfailures 插件：

```
pip install pytest-rerunfailures
```

测试失败后重新运行n次，在重新运行间延迟n秒再运行：

```
# 重新运行3次
pytest --reruns 3 -v -s test_pytest.py
# 重新运行5次，延迟1s
pytest --reruns 5 --reruns-delay 1 -v test_pytest.py
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest --reruns 3 --reruns-delay 1 -v test_pytest.py
========================================== test session starts ==========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.hypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0, remotedata-0.3.2, rerunfailures-9.1
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED                                                    [ 33%]
test_pytest.py::TestDemo::test_answer2 RERUN                                                     [ 66%]
test_pytest.py::TestDemo::test_answer2 RERUN                                                     [ 66%]
test_pytest.py::TestDemo::test_answer2 RERUN                                                     [ 66%]
test_pytest.py::TestDemo::test_answer2 FAILED                                                    [ 66%]
test_pytest.py::TestDemo::test_answer3 PASSED                                                    [100%]

============================================== FAILURES ==============================================
_____________________________________________ TestDemo.test_answer2 _____________________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x00000230ABD58B88>

    def test_answer2(self):
>       assert calc(2, 1) == 4
E       assert 3 == 4
E         +3
E         -4

test_pytest.py:17: AssertionError
========================================== warnings summary ==========================================
test_pytest.py:19
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:19: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - is this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/mark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
====================================== short test summary info ======================================
FAILED test_pytest.py::TestDemo::test_answer2 - assert 3 == 4
================================= 1 failed, 2 passed, 1 warning, 3 rerun in 3.14s =================================
```

4.8 重复执行

安装pytest-repeat插件

```
pip install pytest-repeat
```

重复执行：

```
# 重新执行3次
pytest -v -s --count=3 test_pytest.py
# 设置重复范围：session, module, class或者function（默认）
pytest -v -s --count=3 --repeat-scope=session test_pytest.py
```

或者在代码中标记：

```
import pytest

def calc(a,b):
    return a + b

class TestDemo():
  @pytest.mark.repeat(3)
    def test_answer1(self):
        assert calc(1, 1) == 2
```

4.9 多条断言

一个方法中有多条断言，通常第一条失败后下面就不执行了，pytest-assume插件可以解决断言失败后继续执行断言的问题。

安装

```
pip install pytest-assume
```

执行多条断言:

```
# 写法1
pytest.assume(x == y)
pytest.assume(True)
pytest.assume(False)

# 写法2
with assume: assert calc(2, 1) == 4
with assume: assert calc(2, 1) == 3
with assume: assert calc(2, 2) == 3
```

修改测试用例test_pytest.py:

```
import pytest
from pytest import assume

def calc(a,b):
    return a + b

class TestDemo():
    def test_answer1(self):
        assert calc(1, 1) == 2

    def test_answer2(self):
        with assume: assert calc(2, 1) == 4
        with assume: assert calc(2, 1) == 3
        with assume: assert calc(2, 2) == 3

    @pytest.mark.answer3
    def test_answer3(self):
        assert calc(6, 6) == 12

if __name__=='__main__':
    pytest.main()
```

```
(base) D:\ProgramWorkspace\TestingDemo\test_pytest>pytest -v -s test_pytest.py
========================================== test session starts ==========================================
platform win32 -- Python 3.7.6, pytest-6.0.2, py-1.9.0, pluggy-0.13.1 -- d:\anaconda3\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase('D:\\ProgramWorkspace\\TestingDemo\\test_pytest\\.hypothesis\\examples')
rootdir: D:\ProgramWorkspace\TestingDemo\test_pytest
plugins: allure-pytest-2.8.12, hypothesis-5.5.4, arraydiff-0.3, assume-2.3.2, astropy-header-0.1.2, doctestplus-0.5.0, openfiles-0.4.0, remotedata-0.3.2, rerunfailures-9.1
collected 3 items

test_pytest.py::TestDemo::test_answer1 PASSED
test_pytest.py::TestDemo::test_answer2 FAILED
test_pytest.py::TestDemo::test_answer3 PASSED

============================================== FAILURES ==============================================
_____________________________________________ TestDemo.test_answer2 _____________________________________________

self = <test_pytest.test_pytest.TestDemo object at 0x000002AF5EAECC88>

    def test_answer2(self):
        # assert calc(2, 1) == 4
        with assume: assert calc(2, 1) == 4
        with assume: assert calc(2, 1) == 3
>       with assume: assert calc(2, 2) == 3
E       pytest_assume.plugin.FailedAssumption:
E       2 Failed Assumptions:
E
E       test_pytest.py:20: AssumptionFailure
E       >>      with assume: assert calc(2, 1) == 4
E       AssertionError: assert 3 == 4
E         +3
E         -4
E
E       test_pytest.py:22: AssumptionFailure
E       >>      with assume: assert calc(2, 2) == 3
E       AssertionError: assert 4 == 3
E         +4
E         -3

test_pytest.py:22: FailedAssumption
========================================== warnings summary ==========================================
test_pytest.py:27
  D:\ProgramWorkspace\TestingDemo\test_pytest\test_pytest.py:27: PytestUnknownMarkWarning: Unknown pytest.mark.answer3 - is this a typo?  You can register custom marks to avoid this warning - for details, see https://docs.pytest.org/en/stable/mark.html
    @pytest.mark.answer3

-- Docs: https://docs.pytest.org/en/stable/warnings.html
====================================== short test summary info ======================================
FAILED test_pytest.py::TestDemo::test_answer2 - pytest_assume.plugin.FailedAssumption:
================================== 1 failed, 2 passed, 1 warning in 0.14s ==================================
```

4.10 pytest.main()

除了在终端执行外，也可以通过pytest.main()来执行，pytest.main() 自动查找当前目录下以test*开头的文件或者以*test结尾的py文件。

括号内可传入执行参数，通过[]进行分割，[]内的多个参数通过逗号分割，所有的参数和pytest命令行方式一样：

```
pytest.main(['-v', 'test_pytest.py']) # 执行test_pytest.py用例
```

或者直接在测试文件最后写如下代码，执行py文件。

```
if __name__=='__main__':
    pytest.main()
    # pytest.main(['-v', 'test_pytest.py'])
```

5. pytest 全局配置文件（pytest.ini）

`pytest.ini` 是 pytest 的**全局配置文件**，需放在项目根目录下（文件名固定为`pytest.ini`，参数名也固定），作用是**统一设置 pytest 的执行规则**（无需每次执行命令时重复输入参数）。

```
[pytest]  # 固定开头，标识这是pytest的配置段

# 1. 命令行参数（每次执行pytest时自动附加这些参数）
# 示例：自动添加 `-vs -m smoke`，相当于每次执行`pytest`时默认跑`pytest -vs -m smoke`
addopts = -vs -m smoke
```

```
# 2. 测试用例文件所在目录（pytest会自动去这个目录找用例）
# 示例：指定用例目录为项目根目录下的`testcases`文件夹
testpaths = ./testcases

# 3. 测试文件的匹配规则（pytest只识别符合该规则的文件为测试文件）
# 示例：只识别以`test_`开头、`.py`结尾的文件（如test_login.py）
python_files = test_*.py

# 4. 测试类的匹配规则（pytest只识别符合该规则的类为测试类）
# 示例：只识别以`Test`开头的类（如class TestLogin:）
python_classes = Test*

# 5. 测试函数的匹配规则（pytest只识别符合该规则的函数为测试用例）
# 示例：只识别以`test_`开头的函数（如def test_login():）
python_functions = test_*

# 6. 注册自定义标记（避免pytest报“标记未注册”的警告）
# 示例：注册名为`smoke`的标记，描述为“冒烟测试”
markers =
    smoke: 冒烟测试
```

关键参数补充说明

1. `addopts`：
    - 作用：每次执行`pytest`时，自动附加配置的命令行参数；
    - 示例中`-vs -m smoke`表示：默认显示详细日志 + 打印`print`内容，且只执行带`smoke`标记的用例。
2. `testpaths`：
    - 作用：指定 pytest 查找用例的目录，避免手动输入用例路径；
    - 示例中`./testcases`表示：pytest 自动去项目根目录下的`testcases`文件夹找用例。
3. `markers`：
    - 作用：注册自定义标记（如`smoke`），避免 pytest 执行带标记的用例时抛出 “标记未注册” 的警告；
    - 每个标记占一行，格式为`标记名: 标记描述`。

使用场景

配置`pytest.ini`后，只需在项目根目录执行`pytest`，即可自动按以下规则运行：

- 去`./testcases`目录找用例；
- 只识别`test_*.py`文件、`Test*`类、`test_*`函数；
- 自动附加`-vs -m smoke`参数，执行冒烟测试用例。

这样可以**统一项目的测试执行规则**，避免团队成员每次执行 pytest 时重复输入相同参数。

四，confetst+fixtrue实现WEB自动化的前后置

1. pytest fixture 用法

xUnit style 结构的 fixture用于初始化测试函数， pytest fixture是对传统的 xUnit 架构的setup/teardown功能的改进。pytest fixture为测试准备一个良好的测试环境，测试函数使用的每个fixture通常有一个参数(以 fixture 命名)，测试函数通过参数访问它们。本文将介绍pytest fixture的一些基本用法。

```
import pytest

@pytest.fixture()
def login():
    print("登录")
    return 8

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self, login):
        print("\n开始执行测试用例2")
        print(login)
        assert 2 + login == 10

    def test_case3(self):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100

if __name__ == '__main__':
    pytest.main()
```

test_case2需要调用login方法（或者获取login的返回值），pytest 将会寻找并调用`@pytest.fixture`标记的login() 方法。结果：

```
开始执行测试用例1
PASSED                                   [ 33%]
登录
开始执行测试用例2
8
PASSED                                   [ 66%]
开始执行测试用例3
PASSED                                   [100%]
```

2. 共享 fixture 函数：conftest.py

在测试过程中，多个测试文件可能都要调用 fixture 函数，可以将其移动到 conftest.py 文件中。conftest.py 文件中的 fixture 函数不需要在测试函数中导入，可以被 pytest 自动识别，查找顺序从测试类开始，然后是测试模块，然后是 conftest.py 文件，最后是内置插件和第三方插件。

conftest.py：

```
import pytest
@pytest.fixture()
def login():
    print("登录")
    return 8
```

测试用例：

```
import pytest

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self, login):
        print("\n开始执行测试用例2")
        print(login)
        assert 2 + login == 10

    def test_case3(self):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果：

```
开始执行测试用例1
PASSED                                   [ 33%]
登录
开始执行测试用例2
PASSED                                   [ 66%]
开始执行测试用例3
8
PASSED                                   [100%]
```

3. yield方法

使用yield关键字可以实现setup/teardown的功能，在yield关键字之前的代码在case之前执行，yield之后的代码在case运行结束后执行

```
import pytest

@pytest.fixture()
def login():
    print("登录")
    yield
    print("退出登录")

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self, login):
        print("\n开始执行测试用例2")
        assert 2 + 8 == 10

    def test_case3(self):
```

```
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果:

```
开始执行测试用例1
PASSED                          [ 33%]
登录
开始执行测试用例2
PASSED                          [ 66%]
退出登录
开始执行测试用例3
PASSED                          [100%]
```

4. addfinalizer方法

addfinalizer也可以实现环境的清理，实现与yield方法相同的效果，跟yield不同的是需要注册作为终结器使用的函数。

```
import pytest

@pytest.fixture()
def login(request):
    print("登录")
    def demo_finalizer():
        print("退出登录")
    # 注册demo_finalizer为终结函数
    request.addfinalizer(demo_finalizer)

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self, login):
        print("\n开始执行测试用例2")
        assert 2 + 8 == 10

    def test_case3(self):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果:

```
开始执行测试用例1
PASSED                    [ 33%]
登录
开始执行测试用例2
PASSED                    [ 66%]
退出登录
开始执行测试用例3
PASSED                    [100%]
```

5. fixture 作用范围：Scope

fixture 作用范围可以为module、class、session和function，默认作用域为function。

- **function**：每一个函数或方法都会调用
- **class**：每一个类调用一次
- **module**：每一个.py文件调用一次
- **session**：是多个文件调用一次

```
test_project/
├─ test_module1.py  （module级）
│  ├─ class TestClass1:  （class级）
│  │  ├─ def test_func1():  （function级）
│  │  └─ def test_func2():  （function级）
│  └─ def test_func3():  （function级）
└─ test_module2.py  （module级）
   └─ def test_func4():  （function级）
```

5.1 scope="function"

```python
import pytest

@pytest.fixture(scope="function")
def login():
    print("登录...")

class Test_Demo():
    def test_case1(self, login):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self, login):
        print("\n开始执行测试用例2")
        assert 2 + 8 == 10

    def test_case3(self, login):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100

if __name__ == '__main__':
    pytest.main()
```

结果：

```
登录...
开始执行测试用例1
PASSED                          [ 33%]
登录...
开始执行测试用例2
PASSED                          [ 66%]
登录...
开始执行测试用例3
PASSED                          [100%]
```

5.2 scope="class"

一个class里面多个用例都调用了此fixture，那么只在class里所有用例开始前执行一次

```
import pytest

@pytest.fixture(scope="class")
def login():
    print("登录...")
```

结果：

```
登录...
开始执行测试用例1
PASSED                          [ 33%]
开始执行测试用例2
PASSED                          [ 66%]
开始执行测试用例3
PASSED                          [100%]
```

6. fixture自动应用

6.1 autouse参数

autouse设置为True时，自动调用fixture功能。由于默认作用域为function，不指定scope则每个方法都会调用fixture方法。

```
import pytest

@pytest.fixture(autouse=True)
def login():
    print("登录...")

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self):
        print("\n开始执行测试用例2")
        assert 2 + 8 == 10

    def test_case3(self):
```

```
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果:

```
登录...
开始执行测试用例1
PASSED                    [ 33%]
登录...
开始执行测试用例2
PASSED                    [ 66%]
登录...
开始执行测试用例3
PASSED                    [100%]
```

6.2 @pytest.mark.usefixtures()

在测试方法上加@pytest.mark.usefixtures()

```
import pytest

@pytest.fixture()
def login():
    print("登录...")

@pytest.mark.usefixtures("login")
class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 1 + 1 == 2

    def test_case2(self):
        print("\n开始执行测试用例2")
        assert 2 + 8 == 10

    def test_case3(self):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果:

```
登录...
开始执行测试用例1
PASSED                    [ 33%]
登录...
开始执行测试用例2
PASSED                    [ 66%]
登录...
开始执行测试用例3
PASSED                    [100%]
```

7. fixture函数参数化

如果多条用例都需要调用相同参数，可以将fixture函数参数化。fixture 函数将执行每个参数值，fixture通过固定参数request传递。

```
import pytest

@pytest.fixture(scope="module", params=[
    [1, 1, 2],
    [2, 8, 10],
    [99, 1, 100]
])
def data(request):
    yield request.param

class Test_Demo():
    def test_case1(self):
        print("\n开始执行测试用例1")
        assert 2 + 8 == 10

    def test_case2(self, data):
        print("\n开始执行测试用例2")
        assert data[0] + data[1] == data[2]

    def test_case3(self):
        print("\n开始执行测试用例3")
        assert 99 + 1 == 100


if __name__ == '__main__':
    pytest.main()
```

结果：

```
开始执行测试用例1
PASSED                        [ 20%]
开始执行测试用例2
PASSED                  [ 40%]
开始执行测试用例2
PASSED                  [ 60%]
开始执行测试用例2
PASSED                  [ 80%]
开始执行测试用例3
PASSED                        [100%]
```

注意事项

```python
def test_login(self):
    pass

def test_select_shop(self,all_case_fixture):
    driver = all_case_fixture
    time.sleep(3)
    #进入框架
    driver.switch_to.frame("menu-frame")
    #点击商品列表
    driver.find_element(By.XPATH,"//*[text()=
    # 出框架
    driver.switch_to.default_content()
    # 进入框架
    driver.switch_to.frame("main-frame")
    #操作下拉框
    ele = driver.find_element(By.XPATH,"//sel
    sel = Select(ele)
    #sel.select_by_index(2)
    sel.select_by_value("2")
    #sel.select_by_visible_text("手机类型")
    #点击搜索按钮
```

```python
def all_case_fixture():
    print("打开浏览器，登陆")
    driver = login_ecshop()
    yield driver
    print("关闭浏览器")

def login_ecshop():  1 usage
    # 解决让浏览器不自动关闭
    options = webdriver.ChromeOptions()
    options.add_experimental_option( name: "detach",  valu
    # 创建成浏览器对象
    driver = webdriver.Chrome(options=options)
    #加载项目地址
    driver.get("http://192.168.0.44/ecshop/admin/privile
    # 输入用户名
    driver.find_element(By.XPATH, value: "//input[@name='u
    # 输入密码
    driver.find_element(By.XPATH, value: "//input[@name='p
    # 点击登陆按钮
    driver.find_element(By.XPATH, value: "//input[@value='
    return driver
```

- login_ecshop()用于网站的登录
- login_ecshop()中的return返回值，可用于test_selece_shop的driver值传递
- `yield 后面的内容`：是 Fixture 要**返回给用例的数据**（比如返回浏览器对象`driver`）；

五，POM结构设计

POM（Page Object Model，页面对象模型）是 Selenium 自动化测试中**解耦页面操作与测试逻辑**的设计模式 —— 把每个页面封装成 “对象”，页面的元素和操作（如输入、点击）作为对象的属性 / 方法，测试用例只调用这些方法，不用关心页面细节。

1. 项目结构

```
project/
├── bases/
|   ├── __init__.py
|   ├── base_page.py        # 页面基类
|   └── base_test.py        # 测试基类
├── pages/
|   ├── __init__.py
|   ├── login_page.py       # 具体页面类1
|   ├── home_page.py        # 具体页面类2
|   └── ...                 # 其他页面类
├── tests/
|   ├── __init__.py
|   ├── test_login.py       # 测试用例1
|   └── test_xxx.py         # 其他测试用例
├── utils/                   # 可选
|   └── driver_factory.py   # 浏览器工厂类
└── config/                  # 可选
    └── config.py           # 配置文件
```

2.四层架构

```
┌──────────────────────────────────────────┐
│            POM 四层结构                   │
├──────────────────────────────────────────┤
│                                          │
│  第1层：BasePage（页面基类）               │
│  ├── 封装通用元素操作方法                  │
│  └── 被所有具体页面类继承                  │
│                                          │
│  第2层：具体Page类（如LoginPage）          │
│  ├── 继承BasePage                         │
│  ├── 定义页面元素定位器                    │
│  └── 封装页面业务操作方法                  │
│                                          │
│  第3层：BaseTest（测试基类）               │
│  ├── setUp：测试前初始化（启动浏览器）     │
│  ├── tearDown：测试后清理（关闭浏览器）    │
│  └── 被所有测试类继承                      │
│                                          │
│  第4层：具体测试类（如TestLogin）          │
│  ├── 继承BaseTest                         │
│  ├── 实例化Page对象                       │
│  └── 调用Page方法编写测试逻辑              │
│                                          │
└──────────────────────────────────────────┘
```

3. 简单例子说明

3.1 POM 设计流程--Fixture

第1层：BasePage（基础页）

```
职责：封装所有页面的通用操作
├── 接收 driver（浏览器驱动）
└── 提供通用方法
    └── find_ele()：定位元素（带显式等待）
```

核心代码:

```python
class BasePage:
    def __init__(self, driver):
        self.driver = driver  # 所有页面共享同一个driver

    def find_ele(self, locator):
        # 等待元素出现并返回
        return WebDriverWait(self.driver, 10).until(...)
```

第2层：LoginPage（具体页面）

```
职责：封装登录页的元素和操作
├── 继承 BasePage（获得通用方法）
├── 定义元素定位器（属性）
|    ├── username_loc = "//input[@name='username']"
|    ├── password_loc = "//input[@name='password']"
|    └── submit_loc = "//input[@value='进入管理中心']"
└── 封装业务操作（方法）
     └── login(username, password)：完成登录流程
```

核心代码：

```python
class LoginPage(BasePage):  # 继承BasePage
    # 元素定位器（类属性）
    username_loc = "//input[@name='username']"
    password_loc = "//input[@name='password']"
    submit_loc = "//input[@value='进入管理中心']"

    # 业务方法
    def login(self, username, password):
        self.find_ele(self.username_loc).send_keys(username)  # 调用父类方法
        self.find_ele(self.password_loc).send_keys(password)
        self.find_ele(self.submit_loc).click()
```

第3层：Fixture（测试基础设施）

```
职责：管理浏览器的生命周期
├── 启动浏览器
├── 打开测试页面
├── 传递 driver 给测试用例
└── 测试结束后关闭浏览器
```

核心代码：

```python
@pytest.fixture
def all_case_fixture():
    driver = webdriver.Chrome(...)  # 启动浏览器
    driver.get("登录页URL")
    yield driver  # 把driver传给测试用例
    driver.quit()  # 用例执行完关闭浏览器
```

第4层：TestCase（测试用例）

```
职责：编写测试逻辑
├── 接收 Fixture 传入的 driver
├── 实例化 LoginPage（传入driver）
└── 调用 LoginPage 的方法执行测试
```

核心代码：

```python
class TestShop:
    def test_login(self, all_case_fixture):
        # 实例化登录页
        login_page = LoginPage(all_case_fixture)  # 传入driver

        # 执行登录
        login_page.login("admin", "admin123")

        # 添加断言验证...
```

数据流动示意图

```
┌──────────────────────────────────────────────────────────────┐
│                          执行流程                            │
└──────────────────────────────────────────────────────────────┘

1. Fixture 启动
   └→ 创建 driver ──────────────────────┐
                                        │
2. 测试用例接收 driver                   │
   └→ 实例化 LoginPage(driver) ←────────┘
        └→ 调用父类 BasePage.__init__(driver)
              └→ self.driver = driver 保存

3. 执行 login_page.login("admin", "123")
   └→ 调用 self.find_ele(username_loc)
        └→ 使用 self.driver 定位元素
   └→ 输入用户名
   └→ 输入密码
   └→ 点击登录

4. 测试完成
   └→ Fixture 关闭 driver
```

关键设计要点

层级	作用	为什么这样设计?
BasePage	封装通用操作	避免每个页面类重复写 find_ele、click 等方法
LoginPage	封装页面业务	元素变化时只改这里，测试用例不用动
Fixture	管理浏览器	自动处理启动/关闭，测试用例只关注业务
TestCase	编写测试	代码简洁，像自然语言：`login_page.login(...)`

3.2 POM 设计流程--BaseTest

这是最标准的 POM 设计，用 **BaseTest 替代 Fixture** 的功能。

项目结构

```
项目结构
├── bases/
|   ├── __init__.py
|   ├── base_page.py        # 基础页类
|   └── base_test.py        # 测试基类（新增）
├── poms/
|   ├── __init__.py
|   └── login_page.py       # 登录页类
└── tests/
    └── test_shop.py        # 测试用例
```

---

1. BasePage（不变）

```python
# bases/base_page.py
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.timeout = 10

    def find_ele(self, locator):
        element = WebDriverWait(self.driver, self.timeout).until(
            EC.presence_of_element_located(("xpath", locator))
        )
        return element
```

---

2. BaseTest（新增）- 替代 Fixture

```python
# bases/base_test.py
import unittest
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

class BaseTest(unittest.TestCase):
    """测试基类：管理浏览器生命周期"""

    def setUp(self):
        """每个测试方法执行前：启动浏览器"""
        options = webdriver.ChromeOptions()
        options.add_experimental_option("detach", value=True)

        self.driver = webdriver.Chrome(options=options)
        self.driver.get("http://192.168.0.44/ecshop/admin/privilege.php?
act=logout")

    def tearDown(self):
        """每个测试方法执行后：关闭浏览器"""
        print("关闭浏览器")
        self.driver.quit()
```

---

3. LoginPage（不变）

```python
# poms/login_page.py
from bases.base_page import BasePage

class LoginPage(BasePage):
    username_loc = "//input[@name='username']"
    password_loc = "//input[@name='password']"
    submit_loc = "//input[@value='进入管理中心']"

    def login(self, username, password):
        self.find_ele(self.username_loc).send_keys(username)
        self.find_ele(self.password_loc).send_keys(password)
        self.find_ele(self.submit_loc).click()
```

---

4. 测试用例（继承 BaseTest）

```python
# tests/test_shop.py
import unittest
from bases.base_test import BaseTest  # 继承测试基类
from poms.login_page import LoginPage

class TestShop(BaseTest):  # 继承 BaseTest
    def test_login(self):
        """测试登录"""
        # 使用继承来的 self.driver
        login_page = LoginPage(self.driver)
        login_page.login("admin", "admin123")

        # 断言验证
        self.assertIn("index.php", self.driver.current_url)

if __name__ == '__main__':
    unittest.main()
```

设计流程图

```
┌──────────────────────────────────────────────────┐
│          POM 四层结构（无 Fixture）               │
├──────────────────────────────────────────────────┤
│                                                  │
│  1. BasePage（通用方法）                          │
│      └── find_ele()                              │
│            ↓                                     │
│  2. LoginPage（继承 BasePage）                    │
│      ├── 元素定位器                               │
│      └── login() 方法                            │
│            ↓                                     │
│  3. BaseTest（测试基类）                          │
│      ├── setUp()：启动浏览器 → 创建 self.driver   │
│      └── tearDown()：关闭浏览器                   │
│            ↓                                     │
│  4. TestShop（继承 BaseTest）                     │
│      └── test_login()                            │
│          └── 使用 self.driver 实例化 LoginPage    │
│                                                  │
└──────────────────────────────────────────────────┘
```

核心改动总结

项目	原设计（Fixture）	新设计（BaseTest）
浏览器管理	conftest.py 的 Fixture	BaseTest 的 setUp/tearDown
driver 传递	Fixture 参数注入	继承获得 self.driver
测试类	普通类 + 参数接收	继承 BaseTest / unittest.TestCase

本质相同：都是"在测试前启动浏览器，测试后关闭"，只是实现方式不同：

- **Fixture**：pytest 风格，用装饰器
- **BaseTest**：unittest 风格，用继承

六，数据代码分离及通过Parametrize+Excel实现数据驱动

1. pytest 数据驱动的使用方式

pytest 通过`@pytest.mark.parametrize`实现**数据驱动测试**（用多组数据自动执行同一个用例），核心语法：

python运行

```
@pytest.mark.parametrize(args_name, args_value)
```

- `args_name`：参数名（用例函数接收的参数名，如`caseinfo`）；
- `args_value`：具体测试数据（通常是`list`或`tuple`，列表有多少组数据，用例就自动执行多少次）。

最基本的例子

python运行

```
import pytest

# 用2组数据（"百里"、"北凡"）驱动用例执行2次
@pytest.mark.parametrize("caseinfo", ["百里", "北凡"])
def test_aaa(caseinfo):  # 用例函数接收参数caseinfo
    print("aaa:" + caseinfo)
```

执行结果：用例`test_aaa`会分别输出`aaa:百里`、`aaa:北凡`。

2. 结合 Web 自动化的项目实战（Excel 数据驱动）

通过 Excel 存储测试用例数据，读取后传入用例实现数据驱动，步骤如下：

index	case_name	type	brand	top	supplier
1	根据分类查询商品	4	0	0	0
2	根据分类和品牌组合查询	5	1	0	0

1. 安装第三方依赖库

需安装`pandas`（数据处理）和`openpyxl`（读取 Excel）：bash运行

```
# 使用清华镜像源加速安装
pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple/
pip install openpyxl -i https://pypi.tuna.tsinghua.edu.cn/simple/
```

2.读取 Excel 数据（转为字典列表）

通过`pandas`读取 Excel，将每行数据转为**字典**（键为表头，值为对应单元格内容），最终得到**字典列表**（每组数据对应一个字典）。

```
import pandas  # 导入pandas库

def read_excel_to_list(excel_path, sheet_name):
    """
    读取Excel文件，将数据转为字典列表
    :param excel_path: Excel文件的绝对路径
    :param sheet_name: 要读取的工作表名称
    :return: 字典列表（每个字典对应Excel中的一行数据）
    """
    # 读取Excel（指定engine为openpyxl，支持.xlsx格式）
    data = pandas.read_excel(
        excel_path,
        sheet_name=sheet_name,
        engine='openpyxl'  # 必须指定，否则可能无法读取.xlsx
    )
    # 将DataFrame转为字典列表（orient='records'表示按行转字典）
    dict_list = data.to_dict(orient='records')
    return dict_list


# 调试：直接运行该文件时，验证读取结果
if __name__ == '__main__':
    # 传入Excel文件路径（r表示原始字符串，避免路径中的\被转义）
    # 注意：替换为你实际的Excel文件路径和工作表名称
    dict_list = read_excel_to_list(
        r"D:\workspace\pythonwork\web_frame_3\testcases\sousuo_shop.xlsx",
        "select_shoi"  # 工作表名称（需与Excel中一致）
    )
    # 遍历打印每组数据，查看读取结果
    for data in dict_list:
        print(data)
```

运行上述代码后，控制台会输出：

```
{'index': 1, 'case_name': '根据分类查询商品', 'type': 4, 'brand': 0, 'top': 0,
'supplier': 0}
{'index': 2, 'case_name': '根据分类和品牌组合查询', 'type': 5, 'brand': 1, 'top': 0,
'supplier': 0}
```

（即得到了与 Excel 对应的字典列表，后续可直接传入 pytest 的 `@parametrize` 实现数据驱动）

3.结合 pytest 实现数据驱动

将读取的 Excel 数据传入 `@pytest.mark.parametrize`，驱动 Web 自动化用例：

```python
import pytest
from selenium import webdriver

# 读取Excel数据
test_data = read_excel_data("test_data.xlsx")

@pytest.mark.parametrize("caseinfo", test_data)  # 传入字典列表
def test_search_goods(caseinfo):
    # 初始化浏览器（结合POM可优化）
    driver = webdriver.Chrome()
    driver.get("商品查询页面URL")

    # 使用Excel数据执行操作（示例：输入分类type）
    driver.find_element(By.NAME, "type").send_keys(caseinfo["type"])
    driver.find_element(By.NAME, "brand").send_keys(caseinfo["brand"])
    driver.find_element(By.ID, "search_btn").click()

    # 断言/验证逻辑
    assert caseinfo["case_name"] in driver.page_source

    driver.quit()
```

执行结果：用例会自动执行 2 次（对应 Excel 中的 2 组数据），分别完成 “分类查询” 和 “分类 + 品牌组合查询”。

七，pytest+allure生成美观的测试报告以及实现错误截图

1. Allure 环境安装（下载、解压、配置）

- 下载：从 Allure 官方仓库下载（更稳定，替代原百度网盘链接）：

  官网地址：https://github.com/allure-framework/allure2/releases

  选择对应系统的压缩包（如allure-2.21.0.zip）。
- **解压**：解压到**无中文 / 空格的路径**（如 `D:\allure-2.21.0`）。
- 配置环境变量（PATH）：把 Allure 的bin目录路径（如D:\allure-2.21.0\bin）添加到系统环境变量的PATH中。
- 验证安装：

  打开命令行，执行：bash运行

  ```bash
  allure --version
  ```

若显示版本号（如2.21.0），则安装成功；若提示“命令不存在”，重启 Pycharm / 命令行后重试。

2. 生成 Allure 临时 JSON 报告

在`pytest.ini`中配置 Allure 参数（自动生成临时报告）：

```
[pytest]
# 其他配置...
addopts = -vs --alluredir ./temps --clean-alluredir
# 说明：
# --alluredir ./temps：将测试数据输出到./temps目录（临时JSON报告）
# --clean-alluredir：每次执行前清空temps目录，避免旧数据干扰
```

3. 生成最终 HTML 测试报告

执行测试用例后，通过临时 JSON 报告生成 HTML 报告：在命令行中执行：

```
allure generate ./temps -o ./reports --clean
# 说明：
# ./temps：临时JSON报告目录
# -o ./reports：将HTML报告输出到./reports目录
# --clean：清空reports目录后再生成
```

生成后，打开`./reports/index.html`即可查看美观的 Allure 报告。

补充：实现“错误自动截图”

在 Fixture 中添加“用例失败时自动截图”的逻辑，截图会被 Allure 报告收录：

```
import pytest
import allure
from selenium import webdriver

@pytest.fixture(scope="function")
def driver():
    dr = webdriver.Chrome()
    dr.maximize_window()
    yield dr

    # 后置操作：用例失败时截图
    try:
        # 判断用例是否失败
        if hasattr(pytest, "failed"):
            # 截图并附加到Allure报告
            allure.attach(
                dr.get_screenshot_as_png(),  # 截图二进制数据
                name="用例失败截图",
                attachment_type=allure.attachment_type.PNG
            )
    finally:
        dr.quit()
```

最终效果

执行测试后，Allure 报告将包含：

- 用例执行结果（通过 / 失败 / 跳过）；
- 失败用例的详细错误信息 + 自动截图；
- 用例执行时间、通过率等统计数据。

4. Allure 报告定制：

```
import allure  # 需提前导入allure库
import pytest
from 你的模块 import read_excel_to_list, Shoplist  # 替换为实际模块

# 1. 全局报告定制：Epic（大项目层级的分类）
@allure.epic("项目名称：三日速成最新Pytest+Selenium4的web自动化测试框架训练营")
class TestShop:

    # 2. 数据驱动用例（结合Excel数据）
    @pytest.mark.parametrize(
        "caseinfo",
        read_excel_to_list(r"./testcases/sousuo_shop.xlsx", "select_shop")
    )
    def test_select_shop(self, all_case_fixture, caseinfo):
        # 3. 动态定制报告（基于Excel数据，每个用例显示不同内容）
        allure.dynamic.feature("模块名称：商品管理")  # 报告中“功能模块”层级
        allure.dynamic.story("页面名称：商品列表")      # 报告中“页面/子功能”层级
        allure.dynamic.title("用例名称："+caseinfo["case_name"])  # 报告中“用例标题”（取自Excel的case_name）

        # 测试用例步骤（业务操作，不影响报告定制）
        print(caseinfo)
        sl = Shoplist(all_case_fixture)
        sl.switch_to_frame()
        sl.select_by_value(str(caseinfo["type"]))
        sl.select_brand_by_value(str(caseinfo["brand"]))
        sl.click_search()
```

报告定制代码的作用说明：

代码行	定制项	报告中对应的显示位置
`@allure.epic(...)`	项目层级分类	Allure 报告的 “Epic” 模块（大项目标签）
`allure.dynamic.feature(...)`	功能模块分类	报告的 “Feature” 层级（如 “商品管理” 模块）
`allure.dynamic.story(...)`	子功能 / 页面分类	报告的 “Story” 层级（如 “商品列表” 页面）
`allure.dynamic.title(...)`	用例标题	报告中每个用例的名称（动态读取 Excel 的 `case_name`，每个用例显示不同标题）

报告效果：

Allure 报告中会清晰展示：

- 项目名称 → 功能模块 → 页面 → 用例标题的层级结构；
- 每个用例的标题会自动显示 Excel 中对应的`case_name`（如 “根据分类查询商品”）。

补充

接口测试用例格式

	A	B	C	D	E	F	G	H	I	J	K
	序号	说明	关键字	地址	请求头	参数	状态码	响应头	文本	JSON	保存变
	1	访问首页	get	http://47.107.116.139/phpwind/			200		*csrf_token*		csrf_to
	2	登录	post	http://47.107.116.139/phpw	Accept: "applicatio	username: beifanpass	200		*success*		referer